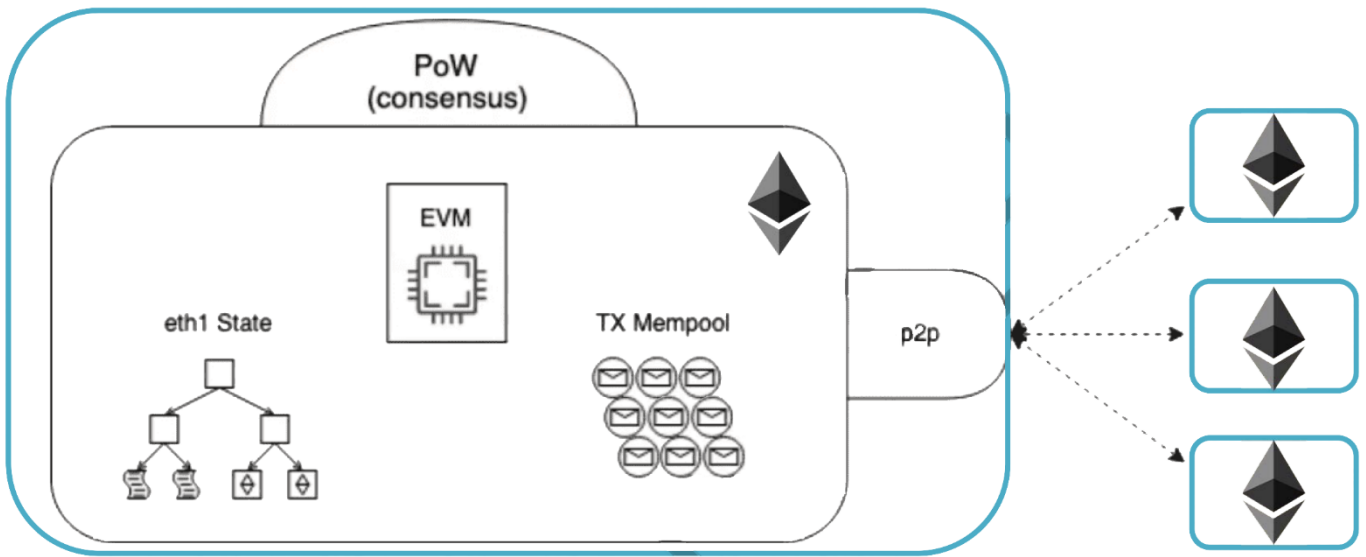


Blockchain and it's Applications

Question Bank Unit 4

1. Draw and explain Ethereum Network.

Ans. 1) Ethereum Network



The Ethereum network is a decentralized, blockchain-based platform designed for the development and execution of smart contracts and decentralized applications (DApps). Here's an explanation of the key components and entities within the Ethereum network:

i) Nodes:

- Nodes are individual computers that participate in the Ethereum network. There are different types of nodes:
 - Full Nodes: Store the entire Ethereum blockchain and participate in the consensus process.
 - Light Nodes: Store a portion of the blockchain and rely on full nodes for transaction validation.
 - Miners: Specialized nodes that participate in the mining process to create new blocks and validate transactions.

ii) Smart Contracts:

- Smart contracts are self-executing programs with code stored on the Ethereum blockchain. They automatically execute predefined rules and actions when specific conditions are met. Smart contracts enable the creation of decentralized applications.

iii) Decentralized Applications (DApps):

- DApps are applications built on top of the Ethereum blockchain. They leverage smart contracts to perform various functions, including financial transactions, identity verification, and more. Examples include decentralized finance (DeFi) applications and decentralized exchanges.

iv) Ether (ETH):

- Ether is the native cryptocurrency of the Ethereum network. It is used for various purposes, including transaction fees (gas), participating in ICOs (Initial Coin Offerings), and as a store of value. Ether is mined by validators through a process called Proof-of-Work (PoW), but Ethereum is transitioning to Proof-of-Stake (PoS) with Ethereum 2.0.

v) Gas:

- Gas is a unit representing the computational effort required to execute operations or smart contracts on the Ethereum network. Users pay gas fees to miners for including their transactions in a block. The concept prevents abuse of the network and ensures fair compensation for computational resources.

vi) Consensus Mechanism:

- Ethereum currently uses Proof-of-Work (PoW) for consensus, where miners compete to solve complex mathematical puzzles to create new blocks and validate transactions. Ethereum 2.0 aims to transition to Proof-of-Stake (PoS) for a more energy-efficient and scalable consensus mechanism.

vii) Blockchain:

- The Ethereum blockchain is a distributed ledger that records all transactions and smart contract executions in a chronological and immutable fashion. It ensures transparency, security, and decentralization by being maintained and updated by a network of nodes.

viii) Ethereum Virtual Machine (EVM):

- The EVM is a runtime environment that executes smart contracts on the Ethereum network. It provides a sandboxed environment, isolating smart contract code from the underlying network, ensuring security and compatibility.

xi) Wallets:

- Wallets in Ethereum store users' private keys and interact with the Ethereum network to send and receive transactions. Wallets can be software-based (online or offline) or hardware-based for enhanced security.

x) Developers and Community:

- Ethereum has a vibrant community of developers, contributors, and users who actively participate in the network's development, improvement proposals (EIPs), and ecosystem expansion.

The Ethereum network is continually evolving, with ongoing upgrades and improvements to address scalability, security, and sustainability concerns. Ethereum 2.0, the next major upgrade, aims to bring significant changes to the consensus mechanism and scalability through the introduction of PoS and shard chains.

2. Discuss DAPPS in Ethereum

Ans. 2) Decentralized Applications, commonly known as DApps, are applications built on blockchain platforms that leverage the decentralized and secure nature of blockchain technology. Ethereum is a popular blockchain platform for developing DApps due to its ability to execute smart contracts, which are self-executing contracts with the terms of the agreement directly written into code. Here are key aspects of DApps in the Ethereum ecosystem:

Characteristics of DApps in Ethereum:

- i. **Decentralization:**
 - DApps operate on a decentralized network of nodes, ensuring that no single entity has control over the entire application. This decentralization enhances security, resilience, and censorship resistance.
- ii. **Smart Contracts:**
 - Smart contracts are the core components of DApps on Ethereum. They are self-executing contracts with code that runs on the Ethereum Virtual Machine (EVM). Smart contracts define the rules and logic of the application, and their execution is secured by the Ethereum blockchain.
- iii. **Open Source:**
 - DApps are often open source, allowing anyone to inspect and contribute to the code. This transparency fosters collaboration, innovation, and community involvement.
- iv. **Token Integration:**
 - Many DApps in Ethereum create and use tokens based on the ERC-20 or ERC-721 token standards. These tokens can represent various assets, such as cryptocurrencies, assets in a game, or ownership rights.
- v. **User Control and Ownership:**
 - DApps enable users to have greater control over their data and assets. Users typically own private keys that grant access to their accounts and assets, reducing reliance on centralized entities.
- vi. **Interoperability:**
 - Ethereum supports interoperability, allowing DApps to interact with each other and share data. This interconnectedness enables the development of a broader ecosystem and facilitates collaboration between different applications.

Examples of DApps:

- i. **Decentralized Finance (DeFi) Apps:**
 - DeFi applications provide financial services without traditional intermediaries. Examples include decentralized exchanges (DEX), lending platforms (e.g., Compound, Aave), and yield farming protocols.
- ii. **Non-Fungible Token (NFT) Platforms:**
 - DApps for creating, buying, and selling non-fungible tokens, which represent unique digital or physical assets. Examples include CryptoKitties, Rarible, and OpenSea.
- iii. **Gaming and Virtual Worlds:**
 - DApps in the gaming industry enable players to own, trade, and sell in-game assets using blockchain technology. Examples include Decentraland, Axie Infinity, and Gods Unchained.
- iv. **Supply Chain Management:**
 - DApps are used for transparent and traceable supply chain management. Products can be tracked from origin to the end consumer, ensuring authenticity and reducing fraud.
- v. **Identity and Authentication:**
 - DApps can provide decentralized identity solutions, allowing users to control and manage their personal information without relying on centralized entities.

Development Stack for Ethereum DApps:

- i. **Smart Contract Development:**
 - Smart contracts are written in languages like Solidity and deployed on the Ethereum blockchain.
- ii. **Frontend Development:**
 - The frontend of the DApp is typically developed using web technologies like HTML, CSS, and JavaScript. Popular frontend frameworks like React or Vue.js are commonly used.
- iii. **Backend Integration:**
 - Depending on the complexity of the DApp, there might be a need for server-side logic or off-chain components. Backend services may interact with the Ethereum blockchain through APIs.
- iv. **Wallet Integration:**
 - DApps often integrate with Ethereum wallets like MetaMask to allow users to interact with the blockchain securely.
- v. **Testing and Deployment:**
 - DApps undergo thorough testing, including unit testing of smart contracts and end-to-end testing of the entire application. Deployment is done on the Ethereum mainnet or testnets.

Developers interested in creating Ethereum DApps can use frameworks like Truffle and development tools like Remix, alongside the Ethereum development environment. The Ethereum ecosystem is dynamic, and ongoing improvements, such as Ethereum 2.0, aim to enhance scalability, security, and sustainability, further supporting the growth of DApps on the platform.

3. Discuss Ethereum Virtual Machine – EVM

Ans. 3) Ethereum Virtual Machine (EVM):

Definition:

- The Ethereum Virtual Machine (EVM) is a runtime environment that executes smart contracts on the Ethereum blockchain. It provides a sandboxed environment for the execution of bytecode generated by compiling smart contract code.

Key Characteristics:

i. Decentralized Execution:

- The EVM is distributed across all Ethereum nodes, ensuring that smart contracts execute consistently on every node in the network.

ii. Turing Complete:

- The EVM is Turing complete, meaning it can theoretically perform any computation that can be expressed algorithmically. This flexibility allows for the implementation of complex smart contracts.

iii. Gas Mechanism:

- The EVM uses a unit called "gas" to measure the computational effort required to execute operations or smart contracts. Gas costs are paid by users in Ether, and they prevent the abuse of computational resources.

iv. Isolation:

- The EVM isolates the execution of smart contracts from the underlying network, ensuring that a contract's execution does not interfere with the state of other contracts or the overall blockchain.

4. Discuss Ethereum Smart Contracts

Ans. 4) Ethereum Smart Contracts

Definition:

- Ethereum Smart Contracts are self-executing contracts with the terms of the agreement directly written into code. These contracts run on the Ethereum blockchain and automatically execute predefined rules and actions when specific conditions are met.

Key Characteristics:

i. Decentralized Execution:

- Smart contracts execute in a decentralized manner across the Ethereum network. No central authority controls or oversees their operation.

ii. Immutable Code:

- Once deployed on the blockchain, the code of a smart contract is immutable. It cannot be altered, ensuring the integrity and predictability of contract execution.

iii. Decentralized Applications (DApps):

- Smart contracts serve as the backbone of decentralized applications (DApps) on the Ethereum platform, enabling the creation of various decentralized services.

iv. Transparent and Trustless:

- The code and logic of smart contracts are transparent and visible on the blockchain. Users can independently verify the contract's functionality, promoting trustlessness.

v. Token Standards:

- Many tokens on the Ethereum blockchain, such as ERC-20 and ERC-721 tokens, are implemented as smart contracts. These standards define the rules for token creation and transfer.

5. Discuss Solidity

Ans. 5) Solidity

Definition:

- Solidity is a high-level programming language specifically designed for writing smart contracts on the Ethereum platform. It is statically typed and supports object-oriented programming principles.

Key Characteristics:

1. Smart Contract Development:

- Solidity is the primary language used for developing Ethereum smart contracts. Developers write code in Solidity, which is then compiled into bytecode for execution on the EVM.

2. Syntax Similarities:

- Solidity's syntax is similar to that of JavaScript and C++, making it accessible to a broad range of developers with experience in these languages.

3. Security Considerations:

- Solidity includes features and best practices for enhancing the security of smart contracts. Developers need to be mindful of potential vulnerabilities, such as reentrancy attacks and integer overflow/underflow.

4. Integration with Ethereum Tooling:

- Solidity integrates seamlessly with Ethereum development tools like Truffle and Remix, providing a comprehensive environment for writing, testing, and deploying smart contracts.

5. Standard Libraries:

- Solidity comes with standard libraries, and developers can leverage existing libraries and frameworks to accelerate the development of secure and efficient smart contracts.

6. Explain and list the part of EVM Memory in blockchain.

Ans. 6) The Ethereum Virtual Machine (EVM) is a crucial component of the Ethereum blockchain network. It is a decentralized virtual machine that enables the execution of smart contracts on the Ethereum platform. The EVM plays a vital role in processing transactions and running decentralized applications (dApps) on the Ethereum network.

Key components of the EVM memory in blockchain include:

i. Stack: The EVM uses a stack-based memory model to store and process data during smart contract execution. The stack is used for temporary storage of data and intermediate results during computation.

ii. Memory: The EVM has a memory area that allows smart contracts to dynamically allocate memory during execution. This memory is used for storing data structures and arrays required by smart contracts.

iii. Storage: The EVM provides persistent storage in the form of key-value pairs for smart contracts. This storage is more permanent compared to memory and is used to store contract state variables and data that need to persist across multiple transactions.

iv. Program Counter (PC): The program counter in the EVM keeps track of the current instruction being executed. It points to the next instruction to be executed and helps in the sequential execution of smart contract code.

v. Gas: Gas is a unit used to measure the computational effort required to execute operations on the Ethereum network. Each operation in the EVM consumes a certain amount of gas, and users need to pay gas fees to execute transactions and smart contracts.

By understanding these components of the EVM memory in blockchain, developers and users can better comprehend how smart contracts are executed and how data is managed within the Ethereum network.

7. What is Hyperledger fabric? Explain in detail.

Ans. 7) Hyperledger Fabric is an open-source, enterprise-grade permissioned distributed ledger platform that provides a foundation for developing blockchain-based applications, solutions, and products tailored for use within private enterprises. It is a modular blockchain framework designed to offer flexibility, scalability, and security for business applications. Fabric is maintained by the Linux Foundation's Hyperledger project, which aims to advance cross-industry blockchain technologies.

Key features and characteristics of Hyperledger Fabric include:

i. Permissioned DLT Platform: Fabric is a permissioned blockchain network, meaning that participants must be authenticated and authorized to join the network. This allows for greater control over access, privacy, and governance compared to permissionless blockchains like Bitcoin.

ii. Modular Architecture: Fabric's modular architecture enables the customization of various components to suit specific use cases and requirements. It allows for plug-and-play components, such as consensus algorithms and membership services, to be integrated into the network.

iii. Smart Contracts: Fabric supports smart contracts written in general-purpose programming languages like Java, Go, and Node.js. This flexibility allows developers to create complex business logic and execute transactions on the blockchain.

iv. Emphasis on Ledger, Smart Contracts, Consensus, Confidentiality, Resiliency, and Scalability: Fabric focuses on providing a robust and efficient ledger, supporting smart contract execution, implementing consensus mechanisms, ensuring data confidentiality, maintaining network resiliency, and enabling scalability for enterprise applications.

v. Hyperledger Foundation: Fabric is part of the Hyperledger project under the Linux Foundation, which fosters collaboration among industry leaders to develop open-source blockchain technologies for various business sectors.

vi. Continuous Development: Fabric has seen continuous development and improvement since its initial release in July 2017. It has a vibrant community of developers and contributors from various organizations working together to enhance the platform's capabilities.

In conclusion, Hyperledger Fabric is a versatile and powerful blockchain platform designed for enterprise use cases, offering features such as permissioned access, modular architecture, smart contract support, and a focus on scalability and security. It provides a solid foundation for building blockchain applications tailored to the specific needs of businesses in diverse industries.

8. Explain Hyperledger sawtooth.

Ans. 8) Hyperledger Sawtooth is another open-source blockchain platform under the Hyperledger project, maintained by the Linux Foundation. Sawtooth is designed to address the scalability and security challenges faced by distributed ledger technologies, offering a flexible and modular architecture for building and deploying blockchain applications. Here are some key aspects of Hyperledger Sawtooth:

i. Modular Design: Sawtooth features a modular design that allows for the customization of consensus algorithms, permissioning systems, and smart contract execution engines. This modularity enables developers to tailor the platform to specific use cases and requirements.

ii. Consensus Mechanisms: Sawtooth supports multiple consensus algorithms, including Practical Byzantine Fault Tolerance (PBFT), Proof of Elapsed Time (PoET), and others. This flexibility allows network participants to choose the consensus mechanism that best suits their needs.

iii. Smart Contracts: Similar to Hyperledger Fabric, Sawtooth supports smart contracts written in popular programming languages like Python and JavaScript. These smart contracts can be used to define the business logic and rules for transactions on the blockchain.

iv. Scalability: Sawtooth is designed to be highly scalable, with features such as parallel transaction processing and dynamic consensus. This scalability is essential for supporting large-scale enterprise applications with high transaction volumes.

v. Permissioned Network: Sawtooth can be configured as a permissioned network, where participants are authenticated and authorized to access the blockchain. This permissioning model provides greater control over data privacy and network governance.

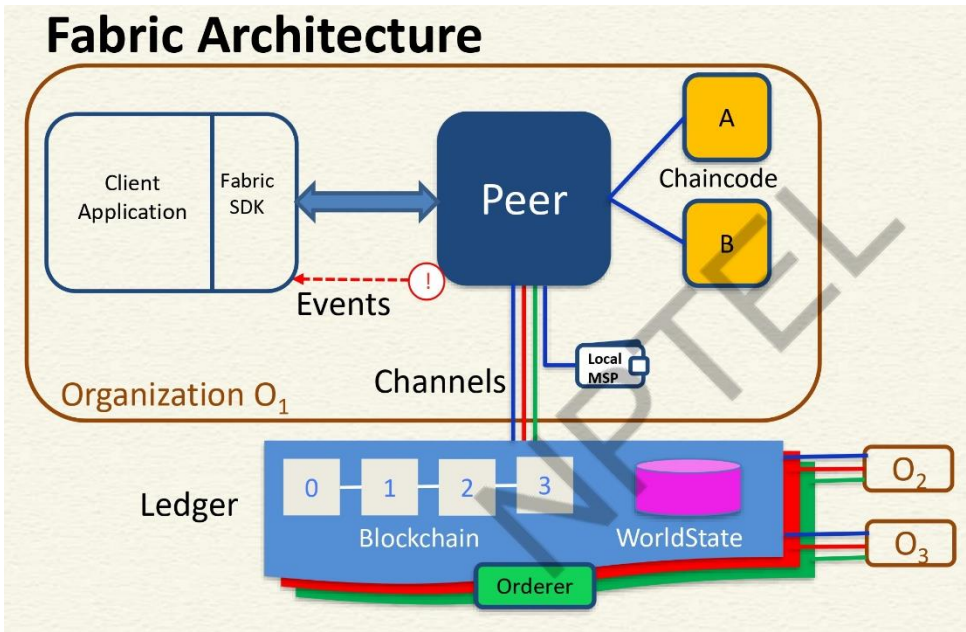
vi. Hyperledger Foundation: Sawtooth is part of the Hyperledger project, which focuses on developing open-source blockchain technologies for enterprise use cases. The project fosters collaboration among industry leaders to advance blockchain adoption across various sectors.

vii. Continuous Development: Sawtooth is actively developed and maintained by a community of contributors from different organizations. Regular updates and enhancements ensure that the platform remains secure, reliable, and up-to-date with the latest blockchain innovations.

In summary, Hyperledger Sawtooth is a versatile and scalable blockchain platform with a modular architecture, support for multiple consensus algorithms, smart contract capabilities, and a focus on enterprise-grade applications. It provides a robust foundation for building and deploying blockchain solutions tailored to the specific needs of businesses in various industries.

9. Draw and explain fabric architecture.

Ans. 9)



The architecture of Hyperledger Fabric is designed to provide a flexible and modular framework for building enterprise-grade blockchain applications. Below is an overview of the key components and their interactions in the Fabric architecture:

- 1. Peer Nodes:** Peer nodes maintain a copy of the ledger and execute transactions. There are two types of peer nodes: endorsing peers, which simulate transactions and create proposals, and committing peers, which validate transactions and update the ledger.
- 2. Ledger:** The ledger consists of two main components: the blockchain and the world state. The blockchain stores a sequential record of transactions, while the world state represents the current state of the ledger after applying all transactions.
- 3. Chaincode:** Chaincode, also known as smart contracts, contains the business logic for processing transactions. It is executed by peer nodes and can be written in general-purpose languages like Java, Go, and Node.js.
- 4. Channels:** Channels are used to partition the network and restrict transaction visibility to specific participants. They enable private and confidential transactions between selected parties on the network.
- 5. Local Membership Service Provider (MSP):** MSP manages the identities of network participants and enforces access control policies. It authenticates users and ensures that only authorized entities can interact with the blockchain network.
- 6. Client Application:** Client applications interact with the blockchain network by submitting transaction proposals, querying the ledger, and receiving transaction results. They communicate with peer nodes to execute transactions.
- 7. Fabric SDK:** The Fabric Software Development Kit (SDK) provides tools and libraries for developers to build applications that interact with the Fabric network. It simplifies the process of integrating blockchain functionality into existing systems.
- 8. Events:** Events are notifications triggered by specific actions on the blockchain, such as the completion of a transaction or the update of the ledger. Clients can subscribe to events to receive real-time updates from the network.
- 9. Orderer Node:** The orderer node is responsible for reaching consensus on the order of transactions and creating blocks to be added to the blockchain. It ensures that transactions are executed in a deterministic and consistent manner across the network.
- 10. Organization (O):** Organizations represent entities that participate in the blockchain network. Each organization may consist of multiple peer nodes and has its own MSP for managing identities and access control.

The Fabric architecture provides a robust foundation for developing scalable, secure, and customizable blockchain applications tailored to the specific requirements of enterprise use cases. By leveraging its modular design and pluggable components, developers can create sophisticated solutions that address a wide range of business challenges.

10. Differentiate between PoW Consensus vs BFT Consensus.

Ans. 10)

Criteria	PoW Consensus	BFT Consensus
Type	Permissionless	Permissioned
Finality	Probabilistic	Deterministic
Scalability	Limited	Better scalability in terms of transaction throughput and confirmation speed
Energy Efficiency	High energy consumption	Lower energy consumption
Security	Highly secure but vulnerable to 51% attacks	Highly secure against Byzantine faults
Speed	Slower due to competition for block creation	Faster due to known participants and quorum requirements
Trust Model	Trustless system with economic incentives	Trust among known participants
Use Cases	Bitcoin and other public blockchains	Enterprise blockchain networks

In summary, Proof of Work (PoW) consensus is typically used in permissionless, public blockchains like Bitcoin, offering high security at the cost of energy consumption and scalability. On the other hand, Byzantine Fault Tolerance (BFT) consensus is commonly employed in permissioned, enterprise blockchains like Hyperledger Fabric, providing faster transaction speeds, lower energy consumption, and deterministic finality with a focus on known participants and trust among them.

Pow vs BFT in detail

Criteria	Proof of Work (PoW) Consensus	Byzantine Fault Tolerance (BFT) Consensus
Type	Permissionless	Permissioned
Participants	Miners compete to solve cryptographic puzzles to validate transactions and create new blocks	Known and trusted nodes participate in the consensus process
Finality	Probabilistic - multiple confirmations are required for transaction finality	Deterministic - once a quorum of nodes agrees on a transaction, it is considered final
Scalability	Limited scalability due to the competitive nature of mining and block creation	Better scalability in terms of transaction throughput and confirmation speed
Energy Efficiency	High energy consumption for solving complex puzzles	Lower energy consumption as there is no need for competitive mining
Security	Highly secure against double-spending attacks but vulnerable to 51% attacks	Highly secure against Byzantine faults and collusion attacks
Speed	Slower due to the time required to solve cryptographic puzzles	Faster due to known participants and quorum requirements for agreement
Trust Model	Trustless system with economic incentives for honest behavior	Trust among known participants with established identities
Use Cases	Public blockchains like Bitcoin, Ethereum	Enterprise blockchain networks like Hyperledger Fabric

In summary, Proof of Work (PoW) consensus is commonly used in permissionless, public blockchains where participants compete to validate transactions, while Byzantine Fault Tolerance (BFT) consensus is prevalent in permissioned, enterprise blockchains where known and trusted nodes collaborate to achieve agreement on transactions efficiently and securely.

11. Explain the blockchain performance triangle.

Ans. 11) The blockchain performance triangle is a concept that illustrates the inherent trade-offs between three fundamental aspects of blockchain systems: decentralization, security, and scalability. Let's delve deeper into each component of the blockchain performance triangle:

i. Decentralization:

Decentralization is a core principle of blockchain technology that aims to distribute control and decision-making across a network of nodes, eliminating the need for a central authority. A decentralized blockchain network offers benefits such as censorship resistance, increased transparency, and enhanced trust among participants. However, achieving high levels of decentralization can lead to challenges such as slower transaction processing speeds, higher resource requirements, and potential scalability issues.

ii. Security:

Security is paramount in blockchain systems to ensure the integrity, confidentiality, and availability of data stored on the ledger. Security measures such as cryptographic algorithms, consensus mechanisms, and network validation protocols are implemented to protect against various threats like double-spending attacks, Sybil attacks, and 51% attacks. Enhancing security in a blockchain network involves robust encryption techniques, secure key management, and resilient consensus algorithms. However, strengthening security measures may introduce complexities that impact scalability and decentralization.

iii. Scalability:

Scalability refers to the ability of a blockchain network to handle a growing number of transactions efficiently without compromising performance. A scalable blockchain can process a high volume of transactions per second, enabling widespread adoption and usability. Achieving scalability involves optimizing network throughput, transaction confirmation times, and block size limits. However, scaling a blockchain network can pose challenges related to network congestion, increased storage requirements, and potential centralization risks.

The blockchain performance triangle highlights the interplay between decentralization, security, and scalability, emphasizing that improvements in one aspect may come at the expense of others. Balancing these three components is crucial for designing a robust and efficient blockchain system that meets the requirements of specific use cases and stakeholders.

Blockchain developers and researchers continuously explore innovative solutions to address the trade-offs within the blockchain performance triangle, seeking to optimize performance while upholding the core principles of decentralization, security, and scalability. By understanding the complexities of the blockchain performance triangle, stakeholders can make informed decisions when designing, implementing, and evaluating blockchain solutions for various applications and industries.

12. What are the function modifiers in solidity?

Ans. 12) In Solidity, function modifiers are special keywords that can be used to modify the behavior of functions in smart contracts. They allow developers to add additional functionality or restrictions to functions without duplicating code. Here are some common function modifiers in Solidity:

i. public: This modifier makes the function accessible from outside the contract and can be called by any account.

ii. external: Functions with the `external` modifier can only be called from outside the contract and not from within the contract itself.

iii. internal: This modifier restricts the function to be called only from within the current contract or contracts deriving from it.

iv. private: Functions marked as `private` can only be called from within the current contract and are not accessible from derived contracts.

v. view: The `view` modifier indicates that the function will not modify the state of the contract. It is used for functions that only read data from the blockchain.

vi. pure: Functions marked as `pure` do not read or modify the state of the contract. They are used for functions that do not interact with the blockchain at all.

vii. payable: The `payable` modifier allows a function to receive Ether along with the function call. This is necessary for functions that need to receive Ether as part of their execution.

viii. Custom Modifiers: Developers can also create custom modifiers to add specific functionality or conditions to functions. Custom modifiers can be defined using the `modifier` keyword followed by the modifier name.

Modifiers in Solidity provide a flexible way to enhance the functionality of functions and enforce certain conditions or restrictions. By using modifiers, developers can improve code readability, reduce redundancy, and ensure the security and efficiency of smart contracts.

13. How is blockchain distributed ledger different from the traditional ledger?

Ans. 13)

Aspect	Traditional Ledger	Blockchain Distributed Ledger
Centralization	Centralized ledger controlled by a single entity or organization	Decentralized ledger shared across a network of computers
Trust	Relies on trust in the central authority maintaining the ledger	Trust is distributed among all participants in the network
Transparency	Limited transparency as access is controlled by the central authority	High transparency as all participants can view the ledger
Immutability	Susceptible to alterations or tampering by the central authority	Immutable ledger where entries are cryptographically secured
Security	Vulnerable to cyber attacks or fraud by the central authority	Enhanced security through consensus mechanisms and cryptography
Efficiency	Transactions may require intermediaries, leading to delays and higher costs	Direct peer-to-peer transactions can be faster and more cost-effective
Flexibility	Limited flexibility in terms of updating or modifying entries	More flexibility through smart contracts and programmable transactions
Scalability	Limited scalability due to centralized control and processing bottlenecks	Potential for greater scalability through distributed processing and consensus algorithms

Blockchain distributed ledger differs from traditional ledger systems in several key ways:

i. Decentralization: Blockchain operates on a decentralized network where multiple nodes validate and record transactions. In contrast, traditional ledgers are centralized and maintained by a single entity.

ii. Transparency: Blockchain ledgers are transparent, allowing all participants to view transaction history. Traditional ledgers may not offer the same level of transparency.

iii. Immutability: Once a transaction is recorded on a blockchain, it cannot be altered or deleted, ensuring the integrity of the ledger. Traditional ledgers may be susceptible to alterations or fraud.

iv. Consensus Mechanism: Blockchain uses consensus mechanisms like Proof of Work or Proof of Stake to validate transactions, while traditional ledgers rely on trust in the central authority.

v. Smart Contracts: Blockchain ledgers can execute smart contracts automatically based on predefined conditions, streamlining processes. Traditional ledgers require manual intervention for contract execution.

vi. Security: Blockchain's cryptographic techniques ensure secure transactions and data integrity, making it more secure than traditional ledgers that may be vulnerable to cyber attacks.

vii. Cost Efficiency: Blockchain eliminates intermediaries and reduces transaction costs compared to traditional ledger systems that involve third parties.

By leveraging these differences, blockchain distributed ledgers offer a more secure, transparent, and efficient way to record and manage transactions compared to traditional ledger systems.